A Version-Controlled SQL Database

Mir Quddus

DASC 6025: Data Cleaning and Quality Assessment

09.25.24



Details of the Tool

TOOL NAME: DOLT

DEVELOPERS/COMPANY: DOLTHUB

LINK: HTTPS://WWW.DO





Introduction

- **Why is Dolt relevant?**
  - The need for version control in databases is growing due to the collaborative nature of data management and the importance of maintaining data integrity and ensuring auditability.
  - Dolt integrates version control directly into SQL, which is critical for tracking data changes, auditing revisions, and ensuring data quality, especially in environments with frequent updates.
- **Why should we care?**
  - As data undergoes frequent changes in fields like finance, healthcare, and collaborative data science projects, ensuring data accuracy, traceability, and audit trails is crucial for compliance and regulatory purposes.
  - Data integrity and auditable histories are cent
    where understanding changes to the dataset
    ensure that the data remains accurate and he
    making.



Related Work



Other Tools in Version Control for Data:

Git: Commonly used for code version control but lacks the ability to handle structured data directly within databases. Git is not designed for tracking data changes within SQL tables, which limits its utility in data-heavy environments.

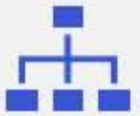


Liquibase: A powerful database schema versioning tool, but it does not track actual data changes. It primarily manages schema evolution and migrations, leaving out the need for detailed data-level versioning.



How Dolt Stands Out:

Dolt integrates both schema and data versioning, providing an all-in-one solution for managing database changes. It tracks changes at the data layer, allowing users to see how data has evolved, while traditional tools like Git or Liquibase only track either code or schema.



Strengths and Weaknesses of Other Tools:

Git: It excels at code version control but doesn't provide features to handle relational database data.

Liquibase is robust in handling schema changes but needs to provide comprehensive data versioning.

Dolt fills this gap by handling both, making it particularly useful in collaborative environments where tracking both schema and data is crucial.

Materials and Methods

Dolt Overview:

Built-in version control at the data layer.

Git-like operations for managing database branches, merging changes, and viewing history.

Commands like dolt diff, dolt merge, dolt checkout are used for data version management.

Methodology for Demonstration:

A practical demo will involve creating a da
data, making
changes, and
trail of data



Installation

Download Dolt:

Windows/Mac: DoltHub.

Linux: sudo apt-get install dolt for Debian-based systems.

Run Installer:

Windows/Mac: Follow the installer prompts.

Linux: Package manager handles installation.

Verify Installation:

All Systems: Open terminal/command prompt and run dolt version.

Set PATH (if needed):

Windows: Add to **System Properties > Path**.

Mac/Linux: Add to .bash_profile or .zshrc.

Initialize Database:

All Sy
the te



Installation Issues

- **PATH Issues:**
- Dolt may not be recognized.
  - Add Dolt to **System Properties > Environment Variables > Path**.
- **DoltPy Issues:**
  - If No module named 'doltpy', install DoltPy using: pip install doltpy
  - Verify Python environment.
- **Visual Studio Code Recommendation:**
  - Instructions may suggest **Visual Studio Code**, but it's not required. Use **Command Prompt** or terminal instead.
- **Recommendation for Improvement:**
  - Suggest providing a **single .exe installer** with all dependencies on Windows.



Demonstration

STEP-BY-STEP DOLT DEMO:

CREATE A DATABASE IN DOLT.

ADD DATA AND SIMULATE CHANGES (E.G., INSERT, UPDATE, DELETE OPERATIONS).

TRACK DATA CHANGES USING DOLT DIFF.

BRANCH AND MERGE DATA CHANGES USING DOLT BRANCH AND DOLT MERGE.

REVERT TO A PREVIOUS VERSION TO DEMONSTRATE HOW VERSION CONTROL ENHANCES DATA QUALITY.



Demo Code

Step 1: Initialize Dolt in Python

```
from doltpy.core import Dolt

# Initialize a new Dolt repository in the current directory
repo = Dolt.init("my_demo_repo")

# Add the repo to the working directory
repo.status()
```

Step 2: Create a New Table

```
# Define table schema
import pandas as pd

table_data = pd.DataFrame({
    'id': [1, 2, 3],
    'name': ['Alice', 'Bob', 'Charlie'],
    'age': [25, 30, 35]
})

# Create the table in the repo
repo.import_df('people', table_data, ['id'], commit=True, message='Initial commit')
```

Step 3: Query the Table

```
# Query the table to see the data
result = repo.sql("SELECT * FROM people")
print(result)
```

```
Query Result:
   id      name  age
0   1     Alice   25
1   2       Bob   30
2   3   Charlie   35
```

Step 4: Commit the Changes

```
# Check the repository status and commit changes
repo.add('people')
repo.commit('Added initial data to peop

Repository Status:
On branch master

Changes to be committed:
  (use "dolt reset HEAD <file>..." 1
    new table:    people
```



Results and Discussion

- **Pros:**
- Version control and collaboration features similar to Git.
- Clear audit trails and data provenance tracking.
- Useful in compliance-heavy sectors like finance and healthcare.

- **Cons:**
- Learning curve for users familiar with traditional SQL but not Git-like systems.
- Performance overhead, especially with large datasets.
- Limited ecosystem compared to older databases like MySQL or PostgreSQL.

- **How Dolt Compares:**
- Outperforms traditional SQL databases in collaborative environments by enabling data version tracking.



Application Areas

FINANCE: ENSURES TRACKING OF ALL CHANGES TO FINANCIAL RECORDS FOR AUDITING.

HEALTHCARE: TRACKS PATIENT DATA CHANGES, ENSURING COMPLIANCE WITH MEDICAL REGULATIONS.

COLLABORATIVE
PROJECTS: ALLOWS D
WORK ON DIFFERENT
SAME DATASET WITH



Related Research

Version Control Systems for Data Science: A Comparative Analysis" (2022):

This article compares Dolt with other systems, noting its unique approach of integrating version control into the data layer.

"Data Provenance in Relational Databases: A Survey and Open Challenges" (2023):

Highlights the importance of data provenance, which Dolt addresses effectively.



Conclusion

- **Summary:**
  - Dolt's integration of version control into SQL databases makes it a powerful tool for ensuring data integrity and quality.
  - Despite its drawbacks (learning curve, performance issues), its potential for collaboration and data auditing is unmatched.
- **Future Implications:**
  - As version control becomes more critical in data management, tools like Dolt will likely see greater adoption in industries requiring stringent compliance and data integrity.



Future Work

BROADER USES OF DOLT: WIDER ADOPTION IN SECTORS REQUIRING STRINGENT DATA AUDITS AND HISTORICAL TRACKING.

POTENTIAL IMPROVEMENTS TO PERFORMANCE AS THE COMMUNITY GROWS AND CONTRIBUTES.



References

Link to DoltHub

The research articles: "Version Control Systems for Data Science: A Comparative Analysis" (2022) and "Data Provenance in Relational Databases: A Survey and Open Challenges" (2023).

Other references used in
section.

